

R, Databases & SQL

Aravind Hebbali

2026-09-26

Table of contents

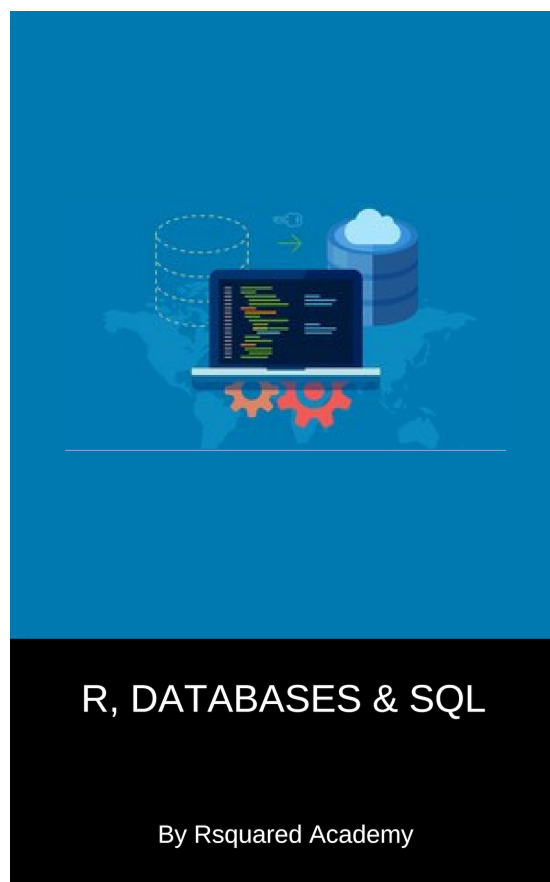
Preface	1
Structure of the book	2
Software information	2
About the Author	5
1 DBI	7
1.1 Introduction	7
1.2 Connection	7
1.2.1 Connection Summary	8
1.2.2 List Tables	8
1.2.3 List Fields	8
1.3 Querying Data	8
1.3.1 Entire Table	9
1.3.2 Few Rows	9
1.3.3 Read Data in Batches	10
1.4 Query	10
1.4.1 Query Status	10
1.4.2 Query Info	10
1.4.3 Latest Query	11
1.4.4 Rows Fetched	11
1.4.5 Rows Affected	11
1.4.6 Column Info	11
1.5 Create Table	12
1.6 Append Data	13
1.7 Insert Rows	14
1.8 Remove Table	15
1.9 SQLite Data Type	15
1.10 Close Connection	15
2 dbplyr	17
2.1 Introduction	17
2.2 Connect to Database	17

2.3	Reference Data	18
2.4	Query Data	18
2.5	Show Query	19
2.6	Collect Data	20
3	SQL Basics	21
3.1	Introduction	21
3.2	Set Up	21
3.3	Select Columns	22
3.3.1	Syntax	22
3.3.2	Select Single Column	22
3.3.3	Select Multiple Columns	23
3.3.4	Select All Columns	23
3.4	Limit	24
3.5	Distinct	24
3.6	Filter	25
3.7	And, Or & Not	27
3.8	BETWEEN	28
3.9	IN	29
3.10	IS NULL	30
3.11	LIKE	30
3.12	Exercises	32
4	SQL Advanced	33
4.1	Introduction	33
4.2	Set Up	33
4.3	Aggregate	34
4.4	Alias	35
4.5	Order By	36
4.6	Group By	38
4.7	Exercises	39
5	JOINS	41
5.1	Introduction	41
5.2	Set Up	41
5.3	INNER JOIN	42
5.4	LEFT JOIN	43
5.5	The SQLite RIGHT / FULL Trap	44
5.6	JOINS with dbplyr	44
5.7	semi_join and anti_join	45
5.8	Exercises	46

Preface

Last verified: 2026-09-26 · R 4.5.2 · DBI · dbplyr 2.6.0 · RSQLite 2.4.6 · duckdb.

New here? Grab the one-page DBI cheatsheet, then work Chapters 1–5 in order — runnable code for each chapter lives in `code/`.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (<http://creativecommons.org/licenses/by-nc-sa/4.0/>).

Structure of the book

Chapter Chapter 1 introduces the **DBI** package. Chapter Chapter 2 explores data wrangling within database using the **dplyr** package. Chapters Chapter 3 and Chapter 4 introduce basic and advanced SQL. Chapter Chapter 5 combines tables with JOINS in SQL and dbplyr.

Software information

The R session information when compiling this book is shown below:

```
sessionInfo()
```

```
R version 4.5.2 (2025-10-31 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 10 x64 (build 19045)
```

```
Matrix products: default
LAPACK version 3.12.1
```

```
locale:
```

```
[1] LC_COLLATE=English_India.utf8 LC_CTYPE=English_India.utf8
[3] LC_MONETARY=English_India.utf8 LC_NUMERIC=C
[5] LC_TIME=English_India.utf8
```

```
time zone: Asia/Calcutta
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
loaded via a namespace (and not attached):
```

```
[1] compiler_4.5.2 fastmap_1.2.0 cli_3.6.5      tools_4.5.2
[5] htmltools_0.5.9 tinytex_0.59  rmarkdown_2.30 knitr_1.50
[9] jsonlite_2.0.0 xfun_0.61     digest_0.6.39 rlang_1.2.0
[13] evaluate_1.0.5
```

We do not add prompts (`>` and `+`) to R source code in this book, and we comment out the text output with two hashes `##` by default, as you can see from the R session information above. This is for your convenience when you want to copy and run the code (the text output will be ignored since it is commented out). Package names are in bold text (e.g., **rmarkdown**), and function names are

followed by parentheses (e.g., `bookdown::render_book()`). The double-colon operator `::` means accessing an object from a package.

About the Author

Aravind Hebbali is the founder of Rsquared Academy. He earned his Masters in Economics from Madras School of Economics. As an active R user, he has authored several R packages such as

- `olsrr`
- `rfm`
- `descriptr`
- `blorr`
- `xplorerr`

In 2015, he founded Rsquared Academy, a free and open source education initiative with focus on data science and analytics. Apart from self paced online courses, Rsquared Academy offers customized learning modules for corporates and universities.

You can find him on [GitHub](#).

Chapter 1

DBI

1.1 Introduction

In this chapter, we will learn to:

- connect to a SQLite database from R
- display database information
- list tables in the database
- query data
 - read entire table
 - read few rows
 - read data in batches
- create table in database
- overwrite table in database
- append data to table in database
- remove table from database
- generate SQL query
- close database connection

We will use the following R packages:

- DBI
- RSQLite

All the data sets used in this chapter can be found [here](#) and code can be downloaded from [here](#) or run from `code/ch1-dbi.R` in this repo.

1.2 Connection

The first step is to connect to a database. We will connect to an in memory SQLite database using `dbConnect()`.

```
con <- dbConnect(RSQLite::SQLite(), ":memory:")
```

NOTE: We use SQLite here because it needs zero setup — no server to install. The exact same DBI commands work with DuckDB, a high-speed analytical engine, for larger analytical queries (see R4DS Ch. 21 at scale):

```
# install.packages("duckdb") # once; high-speed analytical engine
con_duck <- dbConnect(duckdb::duckdb())
dbWriteTable(con_duck, "ecom", ecom)
dbGetQuery(con_duck, "SELECT referrer, AVG(duration) FROM ecom GROUP BY referrer")
```

1.2.1 Connection Summary

We can get the more information about the connection using `summary()`.

```
summary(con)
```

Length	Class	Mode
1	RSQLiteConnection	S4

1.2.2 List Tables

Now that we are connected to a database, let us list all the tables present in it using `dbListTables()`.

```
dbListTables(con)
```

```
[1] "ecom" "sqlite_stat1" "sqlite_stat4"
```

1.2.3 List Fields

Time to explore the `ecom` table in the database. Use `dbListFields()` to list all the fields in the table.

```
dbListFields(con, "ecom")
```

```
[1] "referrer" "device" "bouncers" "n_visit" "n_pages" "duration"
```

1.3 Querying Data

The main objectives of connecting to a database are to:

- query data from the tables already present
- create new tables
- overwrite existing tables
- delete existing tables

Let us begin with querying data. We can do this in the following ways:

- read an entire table at once
- read few rows from a table
- read data in batches

1.3.1 Entire Table

We can read an entire table from a database using `dbReadTable()`.

```
dbReadTable(con, 'ecom')
```

	referrer	device	bouncers	n_visit	n_pages	duration
1	google	laptop	1	10	1	693
2	yahoo	tablet	1	9	1	459
3	direct	laptop	1	0	1	996
4	bing	tablet	0	3	18	468
5	yahoo	mobile	1	9	1	955
6	yahoo	laptop	0	5	5	135
7	yahoo	mobile	1	10	1	75
8	direct	mobile	1	10	1	908
9	bing	mobile	0	3	19	209
10	google	mobile	1	6	1	208
11	direct	laptop	1	9	1	738
12	direct	tablet	0	6	12	132
13	direct	mobile	0	9	14	406
14	yahoo	tablet	0	5	8	80
15	yahoo	mobile	0	7	1	19
16	bing	laptop	1	1	1	995
17	bing	tablet	0	5	16	368
18	google	tablet	1	7	1	406
19	social	tablet	0	7	10	290
20	social	tablet	0	2	1	28

In some cases, we may not need the entire table but only a specific number of rows. Use `dbGetQuery()` and supply a SQL statement specifying the number of rows of data to be read from the table. In the below example, we read ten rows of data from the `ecom` table.

1.3.2 Few Rows

```
dbGetQuery(con, "select * from ecom limit 10")
```

	referrer	device	bouncers	n_visit	n_pages	duration
1	google	laptop	1	10	1	693
2	yahoo	tablet	1	9	1	459
3	direct	laptop	1	0	1	996
4	bing	tablet	0	3	18	468
5	yahoo	mobile	1	9	1	955

6	yahoo laptop	0	5	5	135
7	yahoo mobile	1	10	1	75
8	direct mobile	1	10	1	908
9	bing mobile	0	3	19	209
10	google mobile	1	6	1	208

In case of very large table, we can read data in batches using `dbSendQuery()` and `dbFetch()`. We can mention the number of rows of data to be read while fetching the data using the query generated by `dbGetQuery()`.

1.3.3 Read Data in Batches

```
query <- dbSendQuery(con, 'select * from ecom')
result <- dbFetch(query, n = 15)
result
```

	referrer	device	bouncers	n_visit	n_pages	duration
1	google	laptop	1	10	1	693
2	yahoo	tablet	1	9	1	459
3	direct	laptop	1	0	1	996
4	bing	tablet	0	3	18	468
5	yahoo	mobile	1	9	1	955
6	yahoo	laptop	0	5	5	135
7	yahoo	mobile	1	10	1	75
8	direct	mobile	1	10	1	908
9	bing	mobile	0	3	19	209
10	google	mobile	1	6	1	208
11	direct	laptop	1	9	1	738
12	direct	tablet	0	6	12	132
13	direct	mobile	0	9	14	406
14	yahoo	tablet	0	5	8	80
15	yahoo	mobile	0	7	1	19

1.4 Query

1.4.1 Query Status

To know the status of a query, use `dbHasCompleted()`. It is very useful in cases of queries that might take a long time to complete.

```
dbHasCompleted(query)
```

```
[1] FALSE
```

1.4.2 Query Info

`dbGetInfo()` will return the following:

- the sql statement
- number of rows fetched
- number of rows modified/affected
- status of the query

```
dbGetInfo(query)
```

```
$statement  
[1] "select * from ecom"
```

```
$row.count  
[1] 15
```

```
$rows.affected  
[1] 0
```

```
$has.completed  
[1] FALSE
```

1.4.3 Latest Query

To get the latest query, use `dbGetStatement()`.

```
dbGetStatement(query)
```

```
[1] "select * from ecom"
```

1.4.4 Rows Fetched

To check the number of rows of data returned by a query, use `dbGetRowCount()`.

```
dbGetRowCount(query)
```

```
[1] 15
```

1.4.5 Rows Affected

To know the number of rows modified or affected in the table, use `dbGetRowsAffected()`.

```
dbGetRowsAffected(query)
```

```
[1] 0
```

1.4.6 Column Info

To know the name of the columns and their data types, use `dbColumnInfo()`.

```
dbColumnInfo(query)
```

```
      name      type
1 referrer character
2  device character
3 bouncers  integer
4  n_visit   double
5  n_pages   double
6 duration   double
```

Always clear the result set once you are done fetching and inspecting it.

```
dbClearResult(query)
```

1.5 Create Table

So far we have explored querying data from an existing table. Now, let us turn our attention to creating new tables in the database.

1.5.0.1 Introduction

To create a new table, use `dbWriteTable()`. It takes the following 3 arguments:

- connection name
- name of the new table
- data for the new table

```
x <- 1:10
y <- letters[1:10]
trial <- tibble::tibble(x, y)
dbWriteTable(con, "trial", trial)
```

Let us check if the new table has been created.

```
dbListTables(con)
```

```
[1] "ecom"          "sqlite_stat1" "sqlite_stat4" "trial"
```

```
dbExistsTable(con, "trial")
```

```
[1] TRUE
```

Let us query data from the new table.

```
dbGetQuery(con, "select * from trial limit 5")
```

```
  x y
1 1 a
2 2 b
3 3 c
```

```
4 4 d
5 5 e
```

1.5.0.2 Overwrite Table

In some cases, you may want to overwrite the data in an existing table. Use the `overwrite` argument in `dbWriteTable()` and set it to `TRUE`.

```
x <- sample(100, 10)
y <- letters[11:20]
trial2 <- tibble::tibble(x, y)
dbWriteTable(con, "trial", trial2, overwrite = TRUE)
```

Let us see if the **trial** table has been overwritten.

```
dbGetQuery(con, "select * from trial limit 5")
```

```
  x y
1 99 k
2 30 l
3 34 m
4 24 n
5 81 o
```

1.6 Append Data

You can append data to an existing table by setting the `append` argument in `dbWriteTable()` to `TRUE`.

```
x <- sample(100, 10)
y <- letters[5:14]
trial3 <- tibble::tibble(x, y)
dbWriteTable(con, "trial", trial3, append = TRUE)
```

Let us quickly check if the new data has been appended to the **trial** table.

```
dbReadTable(con, "trial")
```

```
  x y
1 99 k
2 30 l
3 34 m
4 24 n
5 81 o
6 49 p
7 41 q
8 52 r
9 31 s
```

```

10 71 t
11 43 e
12 44 f
13 86 g
14 46 h
15 33 i
16 64 j
17 59 k
18 17 l
19 58 m
20 68 n

```

We can also use `sqlAppendTable()` to append data to an existing table.

```
sqlAppendTable(con, "ecom", head(ecom))
```

```

<SQL> INSERT INTO `ecom`
  (`referrer`, `device`, `bouncers`, `n_visit`, `n_pages`, `duration`)
VALUES
  ('google', 'laptop', TRUE, 10, 1, 693),
  ('yahoo', 'tablet', TRUE, 9, 1, 459),
  ('direct', 'laptop', TRUE, 0, 1, 996),
  ('bing', 'tablet', FALSE, 3, 18, 468),
  ('yahoo', 'mobile', TRUE, 9, 1, 955),
  ('yahoo', 'laptop', FALSE, 5, 5, 135)

```

1.7 Insert Rows

1.7.0.1 Introduction

We can insert new rows into existing tables using:

- `dbExecute()`
- `dbSendStatement()`

Both the function take 2 arguments:

- connection name
- sql statement

```

# use dbExecute
dbExecute(con,
  "INSERT into trial (x, y) VALUES (32, 'c'), (45, 'k'), (61, 'h')"
)
## [1] 3

# use dbSendStatement
res <- dbSendStatement(con,
  "INSERT into trial (x, y) VALUES (25, 'm'), (54, 'l'), (16, 'y')"
)

```

```
)  
dbClearResult(res)
```

1.8 Remove Table

If you want to delete/remove a table from the database, use `dbRemoveTable()`.

```
dbRemoveTable(con, "trial")
```

1.9 SQLite Data Type

If you want to know the data type, use `dbDataType()`.

```
dbDataType(RSQLite::SQLite(), "a")
```

```
[1] "TEXT"
```

```
dbDataType(RSQLite::SQLite(), 1:5)
```

```
[1] "INTEGER"
```

```
dbDataType(RSQLite::SQLite(), 1.5)
```

```
[1] "REAL"
```

1.10 Close Connection

It is a good practice to close connection to a database when you no longer need to read/write data from/to it. Use `dbDisconnect()` to close the database connection.

```
dbDisconnect(con)
```


Chapter 2

dbplyr

2.1 Introduction

In this chapter, we will learn to query data from a database using dplyr.

We will use the following R packages:

- DBI
- RSQLite
- dbplyr
- dplyr

All the data sets used in this chapter can be found [here](#) and code can be downloaded from [here](#) or run from `code/ch2-dbplyr.R` in this repo.

2.2 Connect to Database

Let us connect to an in memory SQLite database using `dbConnect()`.

```
con <- dbConnect(RSQLite::SQLite(), ":memory:")
```

We will copy the `ecom` data to the database so that we can use it for running dplyr statements. We read the same `ecom` data set used across chapters (1000 rows, 8 columns including `country` and `purchase`) from the local `data/web.csv` snapshot, falling back to the online copy if needed.

```
web_path <- if (file.exists("data/web.csv")) "data/web.csv" else "https://raw.githubusercontent.com/rstudio/ecom/master/data/web.csv"
ecom <- readr::read_csv(web_path, show_col_types = FALSE)
ecom <- dplyr::select(ecom, referrer, device, bouncers, n_visit, n_pages, duration, country, purchase)
dplyr::copy_to(con, ecom)
```

2.3 Reference Data

In order to use dplyr functions, we need to reference the table in the database using `tbl()`.

```
ecom2 <- dplyr::tbl(con, "ecom")
ecom2
```

```
# A query: ?? x 8
# Database: sqlite 3.51.2 [:memory:]
  referrer device bouncers n_visit n_pages duration country purchase
  <chr>    <chr>    <int>   <dbl>   <dbl>   <dbl> <chr>    <int>
1 google  laptop      1      10      1     693 Czech Republic    0
2 yahoo   tablet      1       9      1     459 Yemen              0
3 direct  laptop      1       0      1     996 Brazil             0
4 bing    tablet      0       3     18     468 China              1
5 yahoo   mobile      1       9      1     955 Poland             0
6 yahoo   laptop      0       5      5     135 South Africa      0
7 yahoo   mobile      1      10      1      75 Bangladesh        0
8 direct  mobile      1      10      1     908 Indonesia          0
9 bing    mobile      0       3     19     209 Netherlands       0
10 google mobile      1       6      1     208 Czech Republic    0
# i more rows
```

2.4 Query Data

We will look at some simple examples. Let us start by selecting `referrer`, `device` and `duration` columns from `ecom2`.

```
select(ecom2, referrer, device, duration)
```

```
# A query: ?? x 3
# Database: sqlite 3.51.2 [:memory:]
  referrer device duration
  <chr>    <chr>    <dbl>
1 google  laptop    693
2 yahoo   tablet    459
3 direct  laptop    996
4 bing    tablet    468
5 yahoo   mobile    955
6 yahoo   laptop    135
7 yahoo   mobile     75
8 direct  mobile    908
9 bing    mobile    209
10 google mobile    208
# i more rows
```

We can filter data as well. Filter all the rows from `ecom2` where `duration` is greater than 300.

```
filter(ecom2, duration > 300)
```

```
# A query: ?? x 8
# Database: sqlite 3.51.2 [:memory:]
  referrer device bouncers n_visit n_pages duration country purchase
  <chr>    <chr>    <int>    <dbl>    <dbl>    <dbl> <chr>    <int>
1 google  laptop      1        10        1      693 Czech Republic      0
2 yahoo   tablet      1         9        1      459 Yemen                0
3 direct  laptop      1         0        1      996 Brazil               0
4 bing     tablet      0         3       18      468 China                1
5 yahoo   mobile      1         9        1      955 Poland               0
6 direct  mobile      1        10        1      908 Indonesia            0
7 direct  laptop      1         9        1      738 Jamaica               0
8 direct  mobile      0         9       14      406 Ireland              1
9 bing     laptop      1         1        1      995 United States         0
10 bing    tablet      0         5       16      368 Peru                 1
# i more rows
```

Time to do some grouping and summarizing. Let us compute the average time spent on the site for different types of devices.

```
ecom2 %>%
  group_by(device) %>%
  summarise(avg_duration = mean(duration))
```

```
# A query: ?? x 2
# Database: sqlite 3.51.2 [:memory:]
  device avg_duration
  <chr>    <dbl>
1 laptop      376.
2 mobile      337.
3 tablet      354.
```

2.5 Show Query

If you want to view the SQL query generated in the above step, use `show_query()` or `explain()`.

```
avg_time <-
  ecom2 %>%
  group_by(device) %>%
  summarise(avg_duration = mean(duration))

dplyr::show_query(avg_time)
## <SQL>
```

```
## SELECT `device`, AVG(`duration`) AS `avg_duration`
## FROM `ecom`
## GROUP BY `device`

dplyr::explain(avg_time)
## <SQL>
## SELECT `device`, AVG(`duration`) AS `avg_duration`
## FROM `ecom`
## GROUP BY `device`
##
## <PLAN>
##   id parent notused          detail
## 1  6      0      115          SCAN ecom
## 2  8      0          0 USE TEMP B-TREE FOR GROUP BY
```

2.6 Collect Data

Now, some interesting facts. When working with databases, **dplyr** never pulls data into R unless you explicitly ask for it. In the previous example, dplyr will not do anything until you ask for the `avg_time` data. It generates the SQL and only pulls down a few rows when you try to print `avg_time`. So how do we pull all the data and store it for further analysis? `collect()` will pull all the data and store it in a tibble and you can use it for any further analysis.

```
dplyr::collect(avg_time)
```

```
# A tibble: 3 x 2
  device avg_duration
  <chr>    <dbl>
1 laptop    376.
2 mobile    337.
3 tablet    354.
```

It is good practice to close the connection when you are done:

```
dbDisconnect(con)
```

Chapter 3

SQL Basics

3.1 Introduction

In this chapter, we will learn to:

- select
 - single column
 - multiple columns
 - distinct values in a column
- limit the number of records returned
- handle NULL values
- and filter columns using the following operators
 - WHERE
 - AND, or & NOT
 - BETWEEN
 - IN
 - LIKE

We will use the following R packages:

- DBI
- RSQLite
- dbplyr

All the data sets used in this chapter can be found [here](#) and code can be downloaded from [here](#) or run from `code/ch3-sqlbasics.R` in this repo.

3.2 Set Up

In order to be able to use the `ecom` data set for our case study, we will read it using the `readr` package and then copy it to an in memory database using the

`copy_to()` function from **dplyr**.

```
# Canonical ecom: 1000 rows x 8 cols (referrer, device, bouncers, n_visit,
# n_pages, duration, country, purchase), from the local data/web.csv snapshot.
web_path <- if (file.exists("data/web.csv")) "data/web.csv" else "https://raw.githubusercontent.com"
ecom <- readr::read_csv(web_path, show_col_types = FALSE)
ecom <- dplyr::select(ecom, referrer, device, bouncers, n_visit, n_pages, duration, country, purchase)
con <- DBI::dbConnect(RSQLite::SQLite(), ":memory:")
dplyr::copy_to(con, ecom)
```

3.3 Select Columns

The SQL `SELECT` statement is used to fetch the data from a database table.

3.3.1 Syntax

Below is the basic syntax of the `SELECT` statement.

```
SELECT column1, column2, columnN FROM table_name;
```

Here, `column1`, `column2...` are the fields of a table whose values you want to fetch. If you want to fetch all the fields, use the following syntax.

```
SELECT * FROM table_name;
```

3.3.2 Select Single Column

Let us begin by selecting the `device` field from the `ecom` table.

```
SELECT device FROM ecom
```

Table 3.1: Displaying records 1 - 10

device
laptop
tablet
laptop
tablet
mobile
laptop
mobile
mobile
mobile

3.3.3 Select Multiple Columns

Select the following fields from the `ecom` table:

- `referrer`
- `device`
- `purchase`

```
SELECT referrer, device, purchase FROM ecom
```

Table 3.2: Displaying records 1 - 10

referrer	device	purchase
google	laptop	0
yahoo	tablet	0
direct	laptop	0
bing	tablet	1
yahoo	mobile	0
yahoo	laptop	0
yahoo	mobile	0
direct	mobile	0
bing	mobile	0
google	mobile	0

3.3.4 Select All Columns

Select all the fields from the `ecom` table.

```
SELECT * FROM ecom
```

Table 3.3: Displaying records 1 - 10

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
google	laptop	1	10	1	693	Czech Republic	0
yahoo	tablet	1	9	1	459	Yemen	0
direct	laptop	1	0	1	996	Brazil	0
bing	tablet	0	3	18	468	China	1
yahoo	mobile	1	9	1	955	Poland	0
yahoo	laptop	0	5	5	135	South Africa	0
yahoo	mobile	1	10	1	75	Bangladesh	0

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
direct	mo- bile	1	10	1	908	Indonesia	0
bing	mo- bile	0	3	19	209	Netherlands	0
google	mo- bile	1	6	1	208	Czech Republic	0

3.4 Limit

If you have a large table with thousands of rows, returning all the records will take time. Use `LIMIT` to specify the number of records to return.

```
SELECT * FROM ecom limit 10
```

Table 3.4: Displaying records 1 - 10

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
google	lap- top	1	10	1	693	Czech Republic	0
yahoo	tablet	1	9	1	459	Yemen	0
direct	lap- top	1	0	1	996	Brazil	0
bing	tablet	0	3	18	468	China	1
yahoo	mo- bile	1	9	1	955	Poland	0
yahoo	lap- top	0	5	5	135	South Africa	0
yahoo	mo- bile	1	10	1	75	Bangladesh	0
direct	mo- bile	1	10	1	908	Indonesia	0
bing	mo- bile	0	3	19	209	Netherlands	0
google	mo- bile	1	6	1	208	Czech Republic	0

3.5 Distinct

A column in a table may often contain many duplicate values; and we might be interested only in the distinct/unique values. In such cases, we can use the `SELECT DISTINCT` statement to return only distinct values.

```
SELECT distinct referrer FROM ecom
```

Table 3.5: 5 records

referrer
google
yahoo
direct
bing
social

3.6 Filter

Now that we know how to select columns, let us focus on filtering data. In SQL, the WHERE keyword is used to extract only those records that fulfill a specified condition. Data filter based on both text and numeric values in a table. Below are a few comparison operators we can use:

- = equal
- <> not equal
- < less than
- > greater than
- <= less than or equal to
- >= greater than or equal to

The following SQL statement filters all rows from the `ecom` table where the `duration` field is greater than 300.

```
SELECT *
FROM ecom
WHERE duration > 300
```

Table 3.6: Displaying records 1 - 10

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
google	lap- top	1	10	1	693	Czech Republic	0
yahoo	tablet	1	9	1	459	Yemen	0
direct	lap- top	1	0	1	996	Brazil	0
bing	tablet	0	3	18	468	China	1
yahoo	mo- bile	1	9	1	955	Poland	0

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
direct	mo- bile	1	10	1	908	Indonesia	0
direct	lap- top	1	9	1	738	Jamaica	0
direct	mo- bile	0	9	14	406	Ireland	1
bing	lap- top	1	1	1	995	United States	0
bing	tablet	0	5	16	368	Peru	1

Let us filter data based on a text value. In the following example, we filter all rows from the `ecom` table where the `device` used is `mobile`.

```
SELECT *
FROM ecom
WHERE device = 'mobile'
```

Table 3.7: Displaying records 1 - 10

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
yahoo	mo- bile	1	9	1	955	Poland	0
yahoo	mo- bile	1	10	1	75	Bangladesh	0
direct	mo- bile	1	10	1	908	Indonesia	0
bing	mo- bile	0	3	19	209	Netherlands	0
google	mo- bile	1	6	1	208	Czech Republic	0
direct	mo- bile	0	9	14	406	Ireland	1
yahoo	mo- bile	0	7	1	19	France	0
google	mo- bile	1	5	1	147	Brazil	0
bing	mo- bile	0	0	7	196	Russia	0
google	mo- bile	1	10	1	338	Russia	0

3.7 And, Or & Not

The WHERE clause can be combined with other operators such as

- AND - displays a record if all the conditions separated by AND is TRUE
- OR - displays a record if any of the conditions separated by OR is TRUE
- NOT - displays a record if the condition(s) is NOT TRUE

to filter data based on more than one condition or to create more complex conditions.

In the following example, we filter all the rows from the `ecom` table where `n_visit` (visit count) is greater than 3 and `duration` (time spent on the site) is greater than 100. We use AND to create multiple conditions.

```
SELECT *
FROM ecom
WHERE n_visit > 3 AND duration > 100
```

Table 3.8: Displaying records 1 - 10

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
google	laptop	1	10	1	693	Czech Republic	0
yahoo	tablet	1	9	1	459	Yemen	0
yahoo	mobile	1	9	1	955	Poland	0
yahoo	laptop	0	5	5	135	South Africa	0
direct	mobile	1	10	1	908	Indonesia	0
google	mobile	1	6	1	208	Czech Republic	0
direct	laptop	1	9	1	738	Jamaica	0
direct	tablet	0	6	12	132	Estonia	0
direct	mobile	0	9	14	406	Ireland	1
bing	tablet	0	5	16	368	Peru	1

In the next example, we will use both AND & OR. Our goal is to filter all rows from the `ecom` table that follow the below conditions:

- `n_visit` (visit count) is either equal to 3 or 5
- `device` used to visit the website is either `mobile` or `tablet`

```
SELECT *
FROM ecom WHERE (n_visit = 5 OR n_visit = 3)
AND (device = 'mobile' OR device = 'tablet')
```

Table 3.9: Displaying records 1 - 10

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
bing	tablet	0	3	18	468	China	1
bing	mo- bile	0	3	19	209	Nether- lands	0
yahoo	tablet	0	5	8	80	Philip- pines	0
bing	tablet	0	5	16	368	Peru	1
google	mo- bile	1	5	1	147	Brazil	0
yahoo	mo- bile	1	3	1	700	Canada	0
bing	tablet	1	5	1	831	Iran	0
social	tablet	0	3	12	324	China	0
yahoo	tablet	0	3	5	140	Latvia	0
yahoo	tablet	1	3	1	332	France	0

3.8 BETWEEN

The BETWEEN operator selects values within a given range and is inclusive: begin and end values are included. The values can be numbers, text, or dates. In the following example, we filter rows from the `ecom` table where the visit count is between 1 and 3, and the `device` used to visit the website is `mobile`.

```
SELECT *
FROM ecom
WHERE n_visit BETWEEN 1 AND 3 AND device = 'mobile'
```

Table 3.10: Displaying records 1 - 10

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
bing	mo- bile	0	3	19	209	Nether- lands	0
direct	mo- bile	1	2	1	501	El Salvador	0
bing	mo- bile	0	1	1	25	Ireland	0

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
yahoo	mobile	1	3	1	700	Canada	0
direct	mobile	0	1	13	234	Indonesia	0
social	mobile	0	1	2	44	France	0
bing	mobile	0	3	18	288	Portugal	1
yahoo	mobile	1	1	1	73	Indonesia	0
google	mobile	0	2	3	69	Bahamas	0
direct	mobile	1	1	1	423	Mexico	0

3.9 IN

The IN operator allows us to specify multiple values in a WHERE clause. It is a shorthand for multiple OR conditions. In the below example, we filter rows from the `ecom` table where the visit count is either 2 or 4 or 6 or 8 or 10. Instead of using multiple OR conditions, we use the IN operator.

```
SELECT *
FROM ecom
WHERE n_visit IN (2, 4, 6, 8, 10)
```

Table 3.11: Displaying records 1 - 10

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
google	laptop	1	10	1	693	Czech Republic	0
yahoo	mobile	1	10	1	75	Bangladesh	0
direct	mobile	1	10	1	908	Indonesia	0
google	mobile	1	6	1	208	Czech Republic	0
direct	tablet	0	6	12	132	Estonia	0
social	tablet	0	2	1	28	Namibia	0
direct	laptop	1	2	1	384	China	0

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
social	laptop	1	4	1	850	Ukraine	0
direct	tablet	0	2	19	342	Japan	1
google	mobile	1	10	1	338	Russia	0

A field with a NULL value is a field with no value. If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value. In the next example, we filter all rows from the `ecom` table where the `device` column has NULL values.

```
SELECT *
FROM ecom
WHERE device IS NULL
```

Table 3.12: 0 records

referrer	device	bounc- ers	n_visit	n_pages	dura- tion	coun- try	pur- chase
----------	--------	---------------	---------	---------	---------------	--------------	---------------

The LIKE operator is used to search for a specific pattern in a column. There are two wildcards used in conjunction with the LIKE operator:

- % : represents zero, one, or multiple characters
- _ : represents a single character

In the following example, we filter all rows from the `ecom` table where the name of the `country` starts with P. We use `%` after P to indicate that it can be followed by any number or type of characters.

```
SELECT *
FROM ecom
WHERE country LIKE 'P%'
```

Table 3.13: Displaying records 1 - 10

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
yahoo	mobile	1	9	1	955	Poland	0
yahoo	tablet	0	5	8	80	Philippines	0
bing	tablet	0	5	16	368	Peru	1
bing	laptop	1	0	1	456	Portugal	0
yahoo	tablet	1	9	1	706	Philippines	0
direct	tablet	1	8	1	814	Philippines	0
social	laptop	0	5	16	176	Panama	0
yahoo	laptop	1	4	1	928	Peru	0
yahoo	tablet	0	3	11	176	Philippines	0
google	laptop	1	9	1	869	Poland	0

Let us look at another example where we filter all rows from the `ecom` table where the name of the country should follow the below conditions:

- name can start with any character
- the second character must be `o`
- it can have any type or number of characters after the second character

```
SELECT *
FROM ecom
WHERE country LIKE '_o%'
```

Table 3.14: Displaying records 1 - 10

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
yahoo	mobile	1	9	1	955	Poland	0
yahoo	laptop	0	5	5	135	South Africa	0
social	tablet	0	7	10	290	Colombia	1

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
yahoo	mo- bile	0	8	9	225	Colombia	0
direct	lap- top	1	8	1	658	Colombia	0
bing	lap- top	1	0	1	456	Portugal	0
yahoo	tablet	0	7	11	264	Somalia	0
google	tablet	0	1	19	304	Colombia	0
google	lap- top	1	9	1	869	Poland	0
social	tablet	1	1	1	566	Portugal	0

```
dbDisconnect(con)
```

3.12 Exercises

1. List the **referrer** and **device** of 5 visits with more than 8 page views (**n_visit** > 8), ordered by longest **duration** first.
2. How many rows are from **tablet** devices in countries starting with P?
3. Which distinct **device** values appear among bouncers (**bouncers** = 1)?

Chapter 4

SQL Advanced

4.1 Introduction

In this chapter, we will learn to

- aggregate data
- order data
- and group data

We will use the following R packages:

- DBI
- RSQLite
- dbplyr

All the data sets used in this chapter can be found [here](#) and code can be downloaded from [here](#) or run from `code/ch4-sqladvanced.R` in this repo.

4.2 Set Up

In order to be able to use the `ecom` data set for our case study, we will read it using the `readr` package and then copy it to an in memory database using the `copy_to()` function from `dbplyr`.

```
# Canonical ecom: 1000 rows x 8 cols (referrer, device, bouncers, n_visit,
# n_pages, duration, country, purchase), from the local data/web.csv snapshot.
web_path <- if (file.exists("data/web.csv")) "data/web.csv" else "https://raw.githubusercontent.com/
ecom <- readr::read_csv(web_path, show_col_types = FALSE)
ecom <- dbplyr::select(ecom, referrer, device, bouncers, n_visit, n_pages, duration, country, purchase)
con <- DBI::dbConnect(RSQLite::SQLite(), ":memory:")
dbplyr::copy_to(con, ecom)
```

4.3 Aggregate

Let us combine the aggregate statements with WHERE statement to filter data.

- SUM() : returns the total sum of a numeric column

```
SELECT SUM(n_visit) FROM ecom
```

Table 4.1: 1 records

SUM(n_visit)
4972

```
SELECT SUM(n_visit)
FROM ecom
WHERE n_visit > 5
```

Table 4.2: 1 records

SUM(n_visit)
3574

- AVG() : returns the average value of a numeric column

```
SELECT AVG(n_visit) FROM ecom
```

Table 4.3: 1 records

AVG(n_visit)
4.972

```
SELECT AVG(n_visit)
FROM ecom
WHERE country LIKE 'P%'
```

Table 4.4: 1 records

AVG(n_visit)
5.079137

- MAX() : returns the largest value of the selected column

```
SELECT MAX(n_visit) FROM ecom
```

Table 4.5: 1 records

MAX(n_visit)
10

```
SELECT MAX(n_visit)
FROM ecom
WHERE device = 'tablet'
```

Table 4.6: 1 records

MAX(n_visit)
10

- MIN() : returns the smallest value of the selected column

```
SELECT MIN(n_visit) FROM ecom
```

Table 4.7: 1 records

MIN(n_visit)
0

```
SELECT MIN(n_visit)
FROM ecom
WHERE duration BETWEEN 600 AND 900
```

Table 4.8: 1 records

MIN(n_visit)
0

4.4 Alias

SQL aliases are used to give a table, or a column in a table, a temporary name. They are often used to make column names more readable. An alias only exists for the duration of the query. Below are a few examples:

```
SELECT AVG(n_visit) AS avg_mobile
FROM ecom
WHERE device = 'mobile'
```

Table 4.9: 1 records

avg_mobile
5.479651

```
SELECT MAX(n_visit) AS max_visit
FROM ecom
```

Table 4.10: 1 records

max_visit
10

```
SELECT MIN(duration) AS min_duration
FROM ecom
```

Table 4.11: 1 records

min_duration
10

4.5 Order By

The ORDER BY keyword is used to sort the records in ascending or descending order. By default, the records are sorted in ascending order. Use the DESC keyword if you want to sort the records in descending order,

```
SELECT *
FROM ecom
ORDER BY country
```

Table 4.12: Displaying records 1 - 10

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
social	laptop	0	8	2	60	Afghanistan	0
yahoo	laptop	0	10	18	180	Afghanistan	0
social	laptop	1	2	1	274	Afghanistan	0
direct	tablet	1	10	1	682	Afghanistan	0
direct	tablet	0	5	18	414	Afghanistan	1
bing	tablet	0	7	2	34	Afghanistan	0
direct	laptop	0	6	6	96	Albania	0
google	laptop	1	5	1	304	Albania	0
bing	mobile	0	2	2	54	Albania	0
yahoo	laptop	0	3	1	23	Albania	0

```
SELECT *
FROM ecom
ORDER BY duration
```

Table 4.13: Displaying records 1 - 10

referrer	device	bouncers	n_visit	n_pages	duration	country	purchase
yahoo	tablet	1	5	1	10	Poland	0
social	laptop	1	1	1	10	Finland	0
yahoo	laptop	0	0	1	12	Indonesia	0
social	mobile	1	2	1	12	Botswana	0
direct	laptop	0	0	1	13	Czech Republic	0
direct	mobile	0	10	1	17	China	0
yahoo	tablet	0	4	1	17	Poland	0
yahoo	mobile	0	7	1	19	France	0

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
bing	lap- top	0	5	1	20	Thailand	0
yahoo	mo- bile	0	6	1	21	Russia	0

```
SELECT *
FROM ecom
ORDER BY n_visit DESC
```

Table 4.14: Displaying records 1 - 10

referrer	de- vice	bounc- ers	n_visit	n_pages	dura- tion	country	pur- chase
google	lap- top	1	10	1	693	Czech Republic	0
yahoo	mo- bile	1	10	1	75	Bangladesh	0
direct	mo- bile	1	10	1	908	Indonesia	0
google	mo- bile	1	10	1	338	Russia	0
google	mo- bile	1	10	1	479	Cameroon	0
direct	mo- bile	0	10	1	17	China	0
direct	tablet	0	10	3	57	China	0
social	mo- bile	0	10	11	242	Argentina	1
direct	mo- bile	1	10	1	675	Indonesia	0
google	tablet	1	10	1	86	China	0

4.6 Group By

The GROUP BY statement is used with aggregate functions (COUNT, MAX, MIN, SUM, AVG) to group the result by one or more columns.

```
SELECT device,
count(*) AS visits
FROM ecom
GROUP BY device
ORDER by visits DESC
```

Table 4.15: 3 records

device	visits
mobile	344
tablet	331
laptop	325

```
SELECT device, MAX(duration) AS max_duration
FROM ecom
GROUP BY device
ORDER by max_duration DESC
```

Table 4.16: 3 records

device	max_duration
tablet	999
laptop	997
mobile	994

```
dbDisconnect(con)
```

4.7 Exercises

1. Compute the average pages per visit (`AVG(n_pages)`) for each `referrer`, ordered highest average first. Which referrer tops the list?
2. What is the longest `duration` among visits that made a purchase (`purchase = 1`)?
3. Among `mobile` visits only, count visits by `country` and show the top 5. Which country is first?

Chapter 5

JOINS

5.1 Introduction

In this chapter, we will learn to:

- combine rows from two tables with `INNER JOIN`
- keep unmatched rows with `LEFT JOIN`
- avoid the SQLite `RIGHT / FULL OUTER JOIN` trap
- do the same joins from R with `dbplyr inner_join()` / `left_join()`
- filter with `semi_join()` and `anti_join()`

We will use the following R packages:

- DBI
- RSQLite
- dbplyr

All the data sets used in this chapter can be found here and code can be run from `code/ch5-joins.R` in this repo.

5.2 Set Up

So far we have worked with a single table. Real databases keep related data in separate tables. Our `data/ecom.sqlite` file holds two small related tables besides `ecom`:

- `customers`: 20 rows, one per customer (`customer_id` primary key, `country`)
- `orders`: 20 rows, one per order (`order_id` primary key, `customer_id` foreign key into `customers`, `amount`)

Six customers (4, 16–20) have placed no orders, and one order points at a `customer_id` (99) that does not exist. These deliberate mismatches make the difference between join types visible.

Let us connect directly to the SQLite file:

```
con <- DBI::dbConnect(RSQLite::SQLite(), "data/ecom.sqlite")
```

A quick look at both tables:

```
SELECT * FROM customers LIMIT 5
```

Table 5.1: 5 records

customer_id	country
1	United States
2	United Kingdom
3	Germany
4	France
5	Canada

```
SELECT * FROM orders LIMIT 5
```

Table 5.2: 5 records

order_id	customer_id	amount
101	1	459.11
102	1	469.80
103	2	157.35
104	3	418.61
105	3	328.04

5.3 INNER JOIN

`INNER JOIN` returns only the rows that match in **both** tables. Customers without orders, and orders without customers, are dropped.

```
SELECT c.customer_id, c.country, o.order_id, o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.customer_id, o.order_id
LIMIT 6
```

Table 5.3: 6 records

customer_id	country	order_id	amount
1	United States	101	459.11
1	United States	102	469.80
2	United Kingdom	103	157.35
2	United Kingdom	118	76.39
3	Germany	104	418.61
3	Germany	105	328.04

How many of the 20 orders survive? Nineteen – order 120 belongs to the non-existent customer 99 and is excluded:

```
SELECT COUNT(*) AS n FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
```

Table 5.4: 1 records

—
n
—
19
—

5.4 LEFT JOIN

LEFT JOIN keeps **every** row from the left table and fills NULL where the right table has no match. This is how you find customers who never ordered:

```
SELECT c.customer_id, c.country, o.order_id, o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY c.customer_id
```

Table 5.5: 6 records

customer_id	country	order_id	amount
4	France	NA	NA
16	Italy	NA	NA
17	Portugal	NA	NA
18	Greece	NA	NA
19	Ireland	NA	NA
20	Belgium	NA	NA

The full left join has 25 rows: 19 matched order rows plus one NULL row for each of the 6 customers without orders.

5.5 The SQLite RIGHT / FULL Trap

Some tutorials teach `RIGHT JOIN` or `FULL OUTER JOIN`. Do not use them with RSQLite: the SQLite versions bundled with RSQLite (including recent ones) reject both with a syntax error such as `near "RIGHT": syntax error`. Postgres and other production databases accept them, so queries written against SQLite will fail silently only when you move them.

The portable workaround is to swap the tables and use `LEFT JOIN`:

- instead of `A RIGHT JOIN B`, write `B LEFT JOIN A`
- instead of `A FULL OUTER JOIN B`, union a `LEFT JOIN` in each direction

When in doubt, `LEFT JOIN` is the only outer join you need for this book.

5.6 JOINS with dbplyr

The same joins work from R without writing SQL. Reference both tables and use the `dbplyr` join verbs:

```
customers <- dbplyr::tbl(con, "customers")
orders <- dbplyr::tbl(con, "orders")

inner_join(customers, orders, by = "customer_id")
```

```
# A query: ?? x 4
# Database: sqlite 3.51.2 [F:\R\ ebooks\rdbsql\data\ecom.sqlite]
  customer_id country      order_id amount
    <int>   <chr>         <int>   <dbl>
1         1 United States    101    459.
2         1 United States    102    470.
3         2 United Kingdom   103    157.
4         2 United Kingdom   118     76.4
5         3 Germany          104    419.
6         3 Germany          105    328.
7         3 Germany          106    269.
8         5 Canada           107    374.
9         5 Canada           119    248.
10        6 Australia        108     84.6
11        7 Brazil           109    335.
12        8 Japan            110    358.
13        9 Mexico           111    240.
14       10 Netherlands       112    365.
15       11 Sweden            113    469.
16       12 Norway            114    143.
17       13 Denmark           115    242.
18       14 Finland           116    471.
```

```
19           15 Poland           117  490.
```

Use `show_query()` to see the SQL dbplyr generates – it is the `INNER JOIN` you just wrote by hand:

```
q_inner <- inner_join(customers, orders, by = "customer_id")
dplyr::show_query(q_inner)
## <SQL>
## SELECT `customers`.*, `order_id`, `amount`
## FROM `customers`
## INNER JOIN `orders`
##   ON (`customers`.`customer_id` = `orders`.`customer_id`)
```

The left join works the same way:

```
q_left <- left_join(customers, orders, by = "customer_id")
dplyr::show_query(q_left)
## <SQL>
## SELECT `customers`.*, `order_id`, `amount`
## FROM `customers`
## LEFT JOIN `orders`
##   ON (`customers`.`customer_id` = `orders`.`customer_id`)
```

5.7 semi_join and anti_join

Two filtering joins have no direct SQL keyword but are everyday tools:

- `semi_join()` keeps rows of `customers` that **have** a match in `orders` (14 customers). The generated SQL uses `WHERE EXISTS`.
- `anti_join()` keeps rows of `customers` that **lack** a match in `orders` (the same 6 customers from the `LEFT JOIN` check above). The generated SQL uses `WHERE NOT EXISTS`.

```
q_semi <- semi_join(customers, orders, by = "customer_id")
dplyr::show_query(q_semi)
## <SQL>
## SELECT `customers`.*
## FROM `customers`
## WHERE EXISTS (
##   SELECT 1 FROM `orders`
##   WHERE (`customers`.`customer_id` = `orders`.`customer_id`)
## )
dplyr::collect(q_semi)
## # A tibble: 14 x 2
##   customer_id country
##       <int> <chr>
## 1           1    1 United States
```

```
## 2      2 United Kingdom
## 3      3 Germany
## 4      5 Canada
## 5      6 Australia
## 6      7 Brazil
## 7      8 Japan
## 8      9 Mexico
## 9     10 Netherlands
## 10     11 Sweden
## 11     12 Norway
## 12     13 Denmark
## 13     14 Finland
## 14     15 Poland
```

```
q_anti <- anti_join(customers, orders, by = "customer_id")
dplyr::show_query(q_anti)
## <SQL>
## SELECT `customers`.*
## FROM `customers`
## WHERE NOT EXISTS (
##   SELECT 1 FROM `orders`
##   WHERE (`customers`.`customer_id` = `orders`.`customer_id`)
## )
dplyr::collect(q_anti)
## # A tibble: 6 x 2
##   customer_id country
##         <int> <chr>
## 1           4 France
## 2          16 Italy
## 3          17 Portugal
## 4          18 Greece
## 5          19 Ireland
## 6          20 Belgium
```

Rule of thumb: use `semi_join()` / `anti_join()` when you want to filter one table by membership in another without adding the other table's columns.

```
dbDisconnect(con)
```

5.8 Exercises

1. List the `customer_id` and `country` of customers who placed no orders. Write it both as a `LEFT JOIN ... IS NULL` query and as an `anti_join()`.
2. Compute total order `amount` per customer (highest first) with an `INNER JOIN + GROUP BY`. Which customer spent the most?

3. Find the orphan order: which `order_id` points at a non-existent customer?

